# Variable-only equality substitution for iPRA

A five-rule calculus and a smaller complete normalized checker

Prepared for Vladimir Reshetnikov

November 11, 2026

**Abstract**

We further restrict the equality rule in a computation-normal presentation of full first-order intuitionistic primitive recursive arithmetic. The rule needs only substitution for a free numerical variable; it accepts no term template and performs no arbitrary-term occurrence search. A short construction packages two equations in one existential witness and recovers every unrestricted crossed-transport inference. A proof-translation invariant then shows that even the newest numerical parameter suffices. The resulting Eq/$D$ calculus has five inference patterns. Its native Eq/$Q$/$D$ companion has six and gives the smaller measured implementation. Both are equiconsistent and bi-interpretable with iPRA, with whole numerical domains, actual equality, and the identity numerical composites. The new theorem concerns the equality inference. Computation normalization, the paired binder, and the arithmetic interpretation bridge are inherited and explicitly accounted for. Measurements compare complete standalone checkers; no global minimality or new proof-assistant formalization is claimed.

## Contents

**1 Result, comparison boundary, and what is new** **2**

**2 Equality needs only substitution for a numerical variable** **2**

**3 A further restriction: only the newest parameter is needed** **7**

**4 Computation normalization and the bridge to the existing core** **8**

**5 The two interpretations with full first-order iPRA** **11**

**6 Complete implementations and reproducible comparison** **13**

**7 Further research questions** **15**

**8 Deliverable status** **15**

**A Selected complete native checker** **16**

# 1 Result, comparison boundary, and what is new

The mathematical reduction is

$$\frac{s = t \qquad h(s) = r}{r = h(t)} \quad \rightsquigarrow \quad \frac{x = t \qquad u = v}{v = \mathrm{nf}(u[t/x])}, \qquad x \text{ a numerical variable.}$$

The right-hand rule is ordinary capture-free variable substitution followed by the same computation normalization already required for binder instances. The left-hand rule permits an arbitrary source term and a separately supplied term template. We prove that both calculi derive exactly the same sequents. The stronger refinement fixes $x$ to the newest numerical parameter of the current proof scope.

The comparison is pinned to ProveIt commit

`5d85023b0688e03b75f0ec3a04bb43ef0bb51705`.

The project README at that commit describes the native scalar core, its formalized interpretation into iPRA, and the existing reductions [1]. We also inspected the five archives then in `ConciseIPRA/docs/incoming`. In particular, deleting an Eval case, removing $Q$ in favor of clauses, adopting full computation normal forms, and using deterministic replacement of arbitrary source terms were already represented by the prior packages. They are not attributed to this report.

The distinction from `template_free` is precise. That package replaces all occurrences of an arbitrary term and recovers selected-context congruence through a large raw computation marker. Full normalization erases such markers. Here a numerical eigenvariable supplies the name instead. The restriction to variable sources removes the need for arbitrary-term matching altogether. The main proof goes further than the previously proved exclusion of the literal-zero source.

The contribution is local to this pinned comparison. We do not claim that the naming argument or its general proof-theoretic idea is historically unprecedented. The statement that this exact restricted rule is sufficient is proved below and is the claim on which the deliverable rests.

**How to read the result.** The five-rule presentation gives the cleanest formula language: only equality and $D$. The six-rule companion retains $Q$, which makes its complete goal-directed implementation smaller. Rule count and checker size therefore favor different presentations. The implementation section reports the coordinates separately instead of combining the best coordinates of different programs.

**Scope of the equivalence.** Here *iPRA* means full first-order intuitionistic logic over the primitive recursive functions, with their defining equations, the usual successor axioms, and induction for quantifier-free formulas. This is not merely an equational fragment or a statement about standard natural numbers. All interpretation domains used below are the entire numerical sort. The conclusions include both interpretation composites, not just mutual interpretability or equiconsistency.

# 2 Equality needs only substitution for a numerical variable

The numerical terms are the inherited full computation normal forms of

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Let $N$ denote the full normalizer. It contracts both computation rules

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow h[I(n, a; u.h)/u].$$

All substitutions below avoid capture and are followed by $N$. We use the inherited coherence of normalization with substitution, not an unproved substitution theorem for derivations. Put $1 = S0$, $2 = SS0$, and use the term abbreviation

$$C(v, a, b) := N(I(v, a; u.b)), \qquad u \notin \mathrm{FV}(b).$$

Thus $C(0, a, b) \equiv a$ and $C(Sv, a, b) \equiv b$ are literal canonical-syntax identities.

Formulas have grammar $A ::= (s = t) \mid D_{x,y}A$. Contexts are finite reusable lists of clauses $[A; y.B(y)]$, with $y$ bound only in the consequent and absent from $A$. An ordinary assumption $A$ abbreviates $[H; v.A]$, where $H = (0 = 0)$ and $v$ is unused. Its use follows from reflexivity at zero and clause use.

## 2.1 The complete rule inventory

There are five inference patterns. The only changed one is VSub.

$$\frac{}{\Gamma \vdash t = t}. \tag{Refl}$$

$$\frac{\Gamma \vdash x = t \qquad \Gamma \vdash u = v}{\Gamma \vdash v = N(u[t/x])}, \qquad x \text{ is a free numerical variable.} \tag{VSub}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}, \qquad [A; y.B(y)] \in \Gamma. \tag{Use}$$

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad \Gamma, [B(b, 0); y.B(b, y)] \vdash A}{\Gamma \vdash A}. \tag{E}$$

$$\frac{\Gamma, B(t, \eta) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{I}$$

In E, $b$ is fresh for $\Gamma, A$ and the passive parameters of $B$. In I, $z$ is fresh for $\Gamma, t$ and the passive matrix parameters. Every displayed instance is canonical and well scoped. There are no other logical, computation, equality, successor, or induction rules.

VSub substitutes simultaneously for every free occurrence of $x$ in $u$. It neither deletes $x$ from the numerical context nor substitutes in $\Gamma$ or $v$. The target $t$ may contain $x$: inserted targets are not traversed again. No general term template, selected occurrence, path, composite source, or equality-pattern matching is supplied. The source can be any free numerical variable, not only the most recently introduced one. Bound iterator state variables are not sources.

## 2.2 Structural facts that do not require equality congruence

Weakening, injective renaming of numerical parameters, and ordinary-assumption cut are admissible by induction on derivations. Variable sources remain variables under injective renaming, which commutes with their capture-free substitution. The normalizer commutes with renaming. Such renaming also preserves the introduction mode. Before a weakening or cut, choose new eigenparameters away from the finitely many inserted data. A cut replaces each use of the designated constant clause by the supplied proof. These arguments do not assume arbitrary substitution of numerical terms into derivations.

### 2.3 Numerical cases before equality

For a fresh unused coordinate $w$, abbreviate

$$q_y F(y) := D_{w,y} F(y).$$

**Lemma 2.1** (Two conditional-closure interfaces)**.** *The following rules are admissible using only I, E and Use:*

$$\frac{\Gamma, F(0) \vdash F(Sz)}{\Gamma \vdash q_y F(y)}, \qquad \frac{\Gamma \vdash q_y F(y) \qquad \Gamma \vdash F(0)}{\Gamma \vdash F(t)}.$$

*The first rule has the usual fresh $z$; the second permits every well-scoped term $t$.*

*Proof.* For introduction, use I at witness 1 for the unused $w$ coordinate. Because 1 is not literally zero, I is in its case mode for every matrix root. Its premise is exactly the displayed premise.

For elimination, E opens $[F(0); y.F(y)]$: its numerical witness is unused in $F$. Use with the supplied base and target $t$ gives $F(t)$, and E closes the branch. Freshening is allowed by the structural facts. □

**Corollary 2.2** (Numerical cases)**.** *If $z$ is fresh for $\Gamma$ and the passive parameters of $F$, then*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)}$$

*is admissible for every term $t$.*

*Proof.* Weaken the successor proof by $F(0)$ and use closure introduction. Specialize that closure with the given base. The target $t$ can contain parameters already occurring in $\Gamma$; only the fresh generalized parameter $z$ is excluded from $\Gamma$. □

No equality rule has been used. In particular, neither the repair of the missing atomic case-introduction at witness zero nor a compound envelope is needed for this numerical-case construction.

### 2.4 A fresh name directly recovers every numerical template

Define the parity term by

$$P(y) := N\big(I(y, 0; u.C(u, 1, 0))\big).$$

Only the two closed computations $P(1) \equiv 1$ and $P(2) \equiv 0$ will be used. No range theorem or induction for $P$ is required.

**Theorem 2.3** (Recovered unrestricted crossed transport)**.** *For every numerical template $h(w)$ the rule*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash N(h(s)) = r}{\Gamma \vdash r = N(h(t))}$$

*is admissible. Its simulation uses exactly one instance of VSub.*

*Proof.* Choose $x, y$ fresh for all passive data and form the equation matrix

$$B(x, y) := \big(C(y, 0, C(P(y), x, N(h(x)))) = C(y, 0, C(P(y), t, r))\big).$$

Introduce $D_{x,y}B(x,y)$ using witness $s$. Its successor goal is

$$C(P(Sz), s, N(h(s))) = C(P(Sz), t, r).$$

Apply numerical cases to

$$F(v) := \big(C(v, s, N(h(s))) = C(v, t, r)\big).$$

Its zero instance is literally the given $s = t$ and its successor instance is literally the given $N(h(s)) = r$. Numerical cases at $P(Sz)$ therefore proves the required successor goal. Choose the case eigenparameter fresh for the whole current local context and independent of $z$. This proof ignores the temporary assumption of I, so it works in both introduction modes, including when $s$ is literally zero.

Now use E to open the package at a fresh name $b$. The resulting clause is $[H; y.B(b,y)]$, because $B(b,0) \equiv H$. Use that clause at 1 and at 2, with reflexivity as the premise $H$. The two closed parity computations give, literally in canonical syntax,

$$B(b,1) \equiv \big(N(h(b)) = r\big), \qquad B(b,2) \equiv (b = t).$$

One VSub, with major premise $b = t$ and minor premise $N(h(b)) = r$, yields

$$r = N\big(N(h(b))[t/b]\big) = N(h(t)),$$

where the last displayed equality identifies canonical syntax by normal-substitution coherence. It is not an invocation of an equality rule. The fresh name $b$ occurs neither in the outer context nor in the conclusion, so E closes. □

The entire argument preceding the single VSub uses only the logical rules, reflexivity and literal computation. In particular, it assumes no symmetry, transitivity, congruence, arbitrary numerical substitution of derivations, equality reflection, or induction theorem. Passive occurrences of $s$ in $h$ remain untouched, because $b$ was fresh for $h$ and all other passive data. Normalization may duplicate or erase the distinguished occurrences; the substitution-coherence identity justifies this without counting occurrences.

As immediate derived rules, take $h(w) = w$ to obtain root joining, use root joining and reflexivity for symmetry and transitivity, and use crossed transport with reflexivity to obtain congruence. Two crossed transports recover replacement in both sides of every literal-equation template.

### 2.5 Exact comparison and both interpretation composites

Let $\mathcal{C}_T$ be the inherited fully computation-normal Eq/$D$ clause calculus with crossed transport, and let $\mathcal{C}_V$ be the five-rule calculus above. Their canonical terms, formulas, clauses, introduction modes and eigenvariable conditions are identical.

**Theorem 2.4** (Same complete canonical sequents)**.** *For every canonical formula $A$ and finite canonical clause context $\Gamma$,*

$$\Gamma \vdash_{\mathcal{C}_V} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}_T} A.$$

*Both translations are effective on finite proof trees and are literally the identity on the entire end sequent.*

*Proof.* Forwards, VSub is one crossed transport. Given its source variable $x$, abstract all free occurrences of $x$ in its minor left side $u$ by a fresh template variable $w$, obtaining $K(w)$. Coherence gives $N(K(x)) = u$ and $N(K(t)) = N(u[t/x])$. Crossed transport produces exactly the VSub conclusion, including when $t$ itself contains $x$. Translate all other nodes unchanged.

Backwards, replace each crossed-transport node by the preceding derived construction after recursively translating its premises. Copy all other rules. Each newly chosen name is fresh for the finite current context, conclusion and term data; the fresh names used by the numerical-case construction are chosen away from surrounding names. Hence all E and I freshness conditions remain valid and the original sequent is unchanged. □

General numerical substitution on derivations follows by transfer through this exact comparison. It is not a primitive-tag-preserving operation on $\mathcal{C}_V$: a substitution may turn the variable source of a VSub into a composite term. This is a derived admissibility statement, proved through the recovered unrestricted transport.

Compose the exact comparison with the inherited normalized-clausal equivalence and native/iPRA bi-interpretation. The ordinary realization of $\mathcal{C}_V$ is therefore equiconsistent and bi-interpretable with full first-order iPRA. All numerical domains are whole and equality retains its actual meaning. Both numerical composite graphs remain $x = y$; the formula round trips are those of the inherited comparisons.

**What is reduced.** The primitive equality source is now only a variable, and its action is ordinary numerical substitution, which the binder calculus already requires. The primitive equality inference accepts no additional term template and performs no composite-pattern search. The required freedom of ordinary equality is recovered by introducing, using and eliminating fresh numerical names. This is stronger than merely excluding a literal zero source. Full computation normalization is essential to the short bootstrap presented here: it makes the zero, one and two instances of the naming matrices literal identities before equality congruence is available. The proof deliberately uses logical rules; it claims no corresponding completeness theorem for the isolated equality fragment.

### 2.6 The same reduction in the native six-rule presentation

For completeness, the argument also applies before the clause encoding. Keep canonical scalar terms, and use native formulas

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A$$

with ordinary formula contexts. The six patterns are assumption, reflexivity, VSub, and the following three rules:

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}; \tag{$Q$E}$$

$$\frac{\Gamma \vdash D_{x,y} B(x,y) \qquad \Gamma, Q_y B(b,y) \vdash A}{\Gamma \vdash A}; \tag{$D$E}$$

$$\frac{\Gamma, B(t,\eta) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x,y)}. \tag{$D$I}$$

The mode $\eta$, freshness restrictions and VSub are exactly as above. The $Q$ and $D$ formula constructors are primitive in this presentation; no $Q$-introduction is postulated.

Numerical cases still uses no equality rule: introduce $D_{x,y}F(y)$ at witness 1 with unused $x$, ignoring the temporary $F(0)$ assumption; its $D$E branch supplies $Q_yF(y)$, and $Q$E with the given $F(0)$ supplies $F(t)$. For the naming matrix $B$, $D$I and $D$E work exactly as in the theorem. The two uses of the clause $[H; y.B(b, y)]$ are replaced by two $Q$E applications to the ordinary assumption $Q_yB(b, y)$ and reflexivity $H$. The final single VSub is unchanged. Conversely native VSub is a native crossed transport as above. Thus the six-rule native VSub and crossed-transport presentations prove exactly the same sequents under the identity translation. This avoids any dependence on the clause encoding in the measured native implementation.

**Why the source is an arbitrary free variable.** Restricting VSub to the newest numerical parameter would make direct weakening of an arbitrary existing proof fail: inserting a fresh parameter turns its former newest source into an older variable. The proof above does not assume such a stronger restriction. Allowing any free variable makes the required injective renamings and weakening immediate, while still removing arbitrary composite sources and supplied equality templates.

## 3 A further restriction: only the newest parameter is needed

In this section numerical parameter contexts are finite ordered lists. Every eigenparameter is put at the front of its list. Let $\mathcal{C}_0$ have the five rules of $\mathcal{C}_V$, except that its VSub source must be the front parameter of the current numerical context. In de Bruijn notation that source is $v_0$. This variant requires a different metaproof: primitive-tag-preserving weakening of an arbitrary $\mathcal{C}_0$ derivation is false.

**Definition 3.1.** A derivation with initial numerical parameter list $\Xi$ is *internally sourced* if, at every VSub node, its source parameter was introduced by an E eigenparameter binder strictly inside that derivation, rather than belonging to the initial list $\Xi$. Moreover, that E binder is the most recently opened numerical binder still active at the VSub node; there is no intervening active E or I eigenparameter binder. Thus the ordinary newest-source side condition holds at every VSub node; the definition additionally records the source's origin.

**Lemma 3.2** (Safe structural changes for internally sourced proofs)**.** *An internally sourced $\mathcal{C}_0$ derivation admits injective renaming of its initial numerical parameters into any larger initial parameter list, and weakening by clauses using only those initial parameters. The result is internally sourced relative to the enlarged initial list.*

*Proof.* First freshen the finitely many eigenparameters away from all inserted data. Preserve every internal eigenparameter and its binder, and apply the given injection only to the marked initial parameters. Traverse the proof tree while retaining those original marks: the initial list is not reset when descending into a premise. Equivalently, carry the number and names of active internal binders as an induction parameter. An I or E extension puts its unchanged fresh parameter ahead of all pre-existing parameters. At every VSub node the source is an internal parameter. Its origin in the most recently opened active numerical E binder, and hence its newest-source status, cannot be changed by inserting, permuting or renaming the initial parameters, all of which remain older than that source. Capture-free substitution commutes with injective renaming, normalization is equivariant, and renaming preserves the literal-zero mode test. Clause insertion does not change numerical order. All other checks are unchanged.

Equivalently, in de Bruijn notation, initial-parameter renaming acts only on indices at or above the number of locally active eigenbinders. At any VSub node that number is positive and the source is index zero, so the source index is unchanged. This is not a proof of unrestricted weakening for arbitrary $\mathcal{C}_0$ derivations. □

**Theorem 3.3.** *Every $\mathcal{C}_T$ derivation has an internally sourced $\mathcal{C}_0$ translation with the identical end sequent and initial numerical parameter list.*

*Proof.* Induct on the given $\mathcal{C}_T$ proof tree. Recursively translated premises are internally sourced relative to their own initial parameter lists. Copy Refl, Use, E and I nodes unchanged. Combining the translated premises at these nodes preserves the invariant: a source that was introduced inside a premise is also introduced inside the combined derivation.

For a crossed-transport node, apply the direct joint-name construction from the preceding section to the translated premises. Whenever that construction carries a premise proof into a new numerical scope or adds a local clause, apply the safe structural lemma to that entire premise proof. This is legitimate because it is internally sourced by the induction hypothesis. The equality-free numerical-case macro uses only Refl, Use, E and I in addition to these safely weakened premises. Thus no VSub source is created by any of its structural operations.

The construction's one new VSub is at the end of the branch of the fresh-name E binder. Its source $b$ is the newest numerical parameter of that branch. The major and minor premises are obtained from the new clause directly by Use and Refl, so no intervening eigenbinder is active there. Hence the VSub meets the source-zero restriction. Its source was introduced by that E binder inside the constructed proof. No E or I eigenparameter is introduced between that binder and the final VSub node. Thus the entire replacement is internally sourced relative to the original initial list. Close E and complete the induction. □

The reverse translation from $\mathcal{C}_0$ to $\mathcal{C}_T$ is the same local translation of VSub into crossed transport as before. Therefore $\mathcal{C}_0$ and $\mathcal{C}_T$ prove exactly the same sequents. General weakening, arbitrary numerical substitution, and unrestricted crossed transport for arbitrary $\mathcal{C}_0$ premise proofs are now admissible by transferring those proofs to $\mathcal{C}_T$ and translating the resulting proof back. There is no circular use of those structural facts in the completeness proof.

The same invariant and translation work for the native six-rule presentation: replace Use by QE, E by DE, and copy assumption and reflexivity. The direct fresh-name construction's final VSub still occurs under its own newest DE eigenparameter.

# 4 Computation normalization and the bridge to the existing core

The proof above is an exact comparison on canonical sequents. We now make explicit why that comparison retains full iPRA, including the two places where a naive normalization argument would be incomplete.

## 4.1 What the normalizer does

Raw scalar terms are finite expressions $x$, $0$, $S(t)$ and $I(n, a; u.h)$. The binder $u$ binds only in $h$, not in $n$ or $a$. Reduce both computation rules in every numerical subterm:

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow h[I(n, a; u.h)/u].$$

Reduction is allowed inside an iterator body. A canonical term has no iterator whose counter is headed by 0 or $S$. Formula normalization changes terms and preserves equality, $Q$ and $D$ roots. Normalizing a clause treats its consequent under its own bound variable.

The full-normalization package supplies the detailed termination and confluence argument [2]; the following proof records its ingredients used here.

**Lemma 4.1** (Termination, confluence, and substitution coherence)**.** *Every well-scoped scalar term has a unique computation normal form. For capture-free substitution $\sigma$,*

$$\mathrm{nf}(t\sigma) = \mathrm{nf}\big(\mathrm{nf}(t)\ \mathrm{nf}(\sigma)\big).$$

*The same statement holds for formulas and clauses with their binders respected.*

*Proof.* Use a computability argument for ground-type iteration. A body $h(u)$ is computable when substituting any computable numerical term for $u$ produces a strongly normalizing term; passive parameters are handled by a computable valuation. Variables, zero and successor preserve computability.

For iterator closure, suppose $n, a$ are strongly normalizing and $h$ has that substitution property. Let $\nu(t)$ be the maximum reduction length of a strongly normalizing term. Induct lexicographically on

$$\big(\nu(n),\ |n|,\ \nu(a) + \nu(h)\big).$$

A reduction inside the counter decreases the first coordinate. A reduction inside the seed or body decreases the last; the computability property of a reduced body follows by substitution compatibility of reduction. The zero head reduct is $a$. For a successor counter $Sm$, the smaller counter $m$ has the same reduction height and smaller syntax size. The induction hypothesis makes $I(m, a; u.h)$ computable, and the body's substitution property makes the head reduct $h[I(m, a; u.h)/u]$ strongly normalizing. Structural induction on terms under computable valuations completes the argument. Free variables themselves are computable, so open terms are covered.

The two root patterns are disjoint and left linear. Reductions in disjoint subterms commute. A reduction inside a substituted body or argument joins the corresponding duplicated or erased reductions after a head step. This gives local confluence; termination gives confluence. Finally substitution takes a computation reduction to a finite reduction sequence, possibly empty when an erased part is affected. Both sides of the displayed identity reduce from substitution instances of the same raw term, so their unique normal forms agree. □

No theorem that the *uniform* normalizer is primitive recursive is assumed. Every fixed iterator description represents a primitive recursive operation, whereas normalization takes arbitrary descriptions as input. The total normalizer is part of the trusted algorithm and is counted in the source measurements. A practical run may exhaust time or memory on a large certificate. The mathematical calculus permits every finite derivation; the Python programs implement it subject to ordinary resource limits.

### 4.2 Restoring every case introduction

The single introduction I selects progress for an equation at the literal zero witness. A raw witness can become zero under normalization. It is therefore incorrect simply to normalize a raw I node and assume that its mode stayed unchanged.

First work in $\mathcal{C}_T$, where crossed transport is available. For an equation $E = (s = t)$ put

$$G(v, E) := (C(v, 0, s) = C(v, 0, t)), \qquad L(E) := q_v G(v, E) = D_{a,v} G(v, E),$$

with unused $a$ and fresh binders. The zero instance of $G$ is literally $0 = 0$; its successor instance is literally $E$. Hence

$$E \vdash L(E), \qquad L(E) \vdash E.$$

The first direction uses the equality-free case-closure interface at dummy witness 1; the second eliminates that closure at 1 using reflexivity. These conversions use neither equation induction nor equality transport.

Suppose the desired missing case has premise $\Gamma, E(0, 0) \vdash E(0, Sz)$. Wrap its base and conclusion in $L$. The matrix $L(E(x, y))$ has a compound $D$ root, so I at witness zero uses cases and gives $D_{x,y} L(E(x, y))$. Eliminate this at a fresh variable $b$. To introduce $D_{x,y} E(x, y)$ at witness $b$, turn its assumed base $E(b, 0)$ into $L(E(b, 0))$, use the retained clause at $Sz$, and unwrap the result. Witness $b$ is not literally zero, so the final I is again in case mode. Close the elimination.

Thus every case introduction is derivable. Progress for an arbitrary equation $E(y)$ uses I with an unused witness coordinate and witness zero, then the $q$-elimination interface. This supplies contextual equation induction once a base is given.

### 4.3 An auxiliary calculus removes mode ambiguity

Use an auxiliary canonical calculus with separate unrestricted case introduction and equation-progress introduction. Its terms and other rules are unchanged. The preceding constructions translate it into $\mathcal{C}_T$. Conversely a case-mode I is unrestricted case introduction, and a progress-mode I is equation progress followed by case introduction. Thus the two canonical calculi have the same consequences.

Translate the raw native core into the auxiliary canonical calculus: normalize numerical terms, replace $Q_y B$ by $q_y B$ in the clausal presentation, and translate ordinary assumptions to constant clauses. Computation axioms become reflexivities. A raw transport instance becomes a canonical one by normalized-substitution coherence. Closure elimination becomes $q$-elimination. An eliminated $D$ supplies its retained clause, from which the corresponding $q$ assumption can be reintroduced. The case and progress rules are handled separately, so no witness-mode invariance is presumed.

For the reverse comparison, every reduction represents a provable equation in the raw core. Reducing inside a seed or counter uses ordinary congruence. Reducing inside a *bound update body* requires the following derived fact, rather than an uncharged function-extensionality axiom.

**Lemma 4.2** (Iterator-body extensionality in the raw theory)**.** *If the raw theory proves $h(u) = k(u)$ with $u$ fresh for the retained context and passive parameters, it proves*

$$I(n, a; u.h) = I(n, a; u.k).$$

*Proof.* Apply literal-equation induction to equality of the two trajectories. Their initial values are both $a$. At the successor step use their computation equations, the induction hypothesis, and the proved pointwise body equality. All passive parameters stay fixed. □

This proves $t = \mathrm{nf}(t)$ for each raw term, including reductions inside update bodies, and lifts to formula and clause equivalence. It completes the raw/canonical comparison. The inherited written source arguments are included under `reference/`.

**Theorem 4.3** (Faithful comparison with the native core)**.** *The native scalar core and $\mathcal{C}_T$ translate proofs in both directions. Both formula composites are internally interderivable. The numerical maps are normalization and raw inclusion, whose composites are internally equal to the identity. The same holds for $\mathcal{C}_V$ and $\mathcal{C}_0$ by their exact equality-rule comparisons.*

# 5 The two interpretations with full first-order iPRA

## 5.1 Ordinary realization

Let $T_I$ be intuitionistic first-order logic with actual equality over the scalar iterator language, both computation equations, contextual induction for literal equations, coverage and separation:

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The separation sentence is explicit. Positive equations alone admit a singleton model, so the ordinary falsity bridge cannot be silently omitted.

The formula and clause readings are

$$(s = t)^\circ = (s = t), \quad (D_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)),$$

$$[A; y.B]^\circ = A^\circ \to \forall y\, B^\circ(y), \qquad (Q_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y).$$

The interpretation domain is $x = x$; numerical equality is actual identity.

Every rule is sound. Refl is identity reflexivity. VSub follows from equality replacement and symmetry. Use and E are implication, universal, and existential elimination. Case-mode I uses coverage. Progress-mode I uses induction for its literal equation under a temporary zero-instance assumption. The eigenvariable conditions keep the retained context and witness fixed. Every normalization step is already an equation of $T_I$.

## 5.2 The inherited arithmetic compiler

The pinned native development proves $T_I$ bi-interpretable with iPRA [1, 5]. Its arithmetic dependency order is significant: first a scalar predecessor, then reflection of actual equality into a numerical zero test, then quantifier-free induction, then pairing and the primitive-recursive compiler. None is assumed to prove the new variable-source reduction.

In iPRA interpret each fixed iterator by ordinary primitive recursion ignoring the counter in its update:

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Each fixed finite body has its own primitive-recursive description. There is no universal total evaluator on arbitrary description codes.

In the reverse direction the inherited scalar construction supplies pairing and projection terms $\pi, L, H$, with $L(\pi(a, b)) = a$ and $H(\pi(a, b)) = b$ proved internally. For a fixed primitive-recursive definition

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

define

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))),$$
$$f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the projection identities give the recursion equations. Zero, successor, projections and composition translate directly. The construction recurses externally on a finite definition. It does not replace the numerical domain by a coding range.

### 5.3 Full intuitionistic logic and the composites

The native development defines formula selectors $\mathrm{sel}(v, A, B)$ and gates $\mathrm{gate}(v, A)$ by structural recursion on formulas. At zero the selector is $A$, and at a successor it is $B$; the gate is true at zero and equivalent to $A$ at a successor. For equations the definitions apply $C$ to the two endpoints. Compound formulas are padded and their binders aligned before recursing. This is a finite syntactic construction, not quantification over predicates.

With $q_y B = D_{w,y} B$ for an unused $w$, the logical macros are

$$\begin{aligned}
\bot_* &= (0 = S0),\\
A \Rightarrow_* B &= q_y\,\mathrm{sel}(y, A, B),\\
A \wedge_* B &= q_y\,\mathrm{gate}\big(y, \mathrm{sel}(P(y), A, B)\big),\\
\exists_* x\, A(x) &= D_{x,y}\,\mathrm{gate}(y, A(x)),\\
A \vee_* B &= \exists_* v\ \mathrm{sel}(v, A, B),\\
\forall_* x\, A(x) &= A(0) \wedge_* q_x A(x).
\end{aligned}$$

Dummy coordinates are fresh, and substitutions avoid capture. The inherited derivations in `reference/inherited_choice_basis.tex` provide their intuitionistic natural-deduction rules. Our exact sequent comparisons preserve those derivations.

For the numerical round trips, two translations of a fixed operation satisfy the same initial and recursion equations. Equation induction identifies their trajectories. External induction on a finite operation description proves that its double translation equals the original operation on *every internal input.* The argument also accounts for changes of staged body descriptions after substitution. Formula structural induction then proves both logical round trips.

**Theorem 5.1** (Bi-interpretability and equiconsistency)**.** *The ordinary realizations of $\mathcal{C}_V$, $\mathcal{C}_0$, and their native six-rule companions are bi-interpretable with full first-order iPRA. Both numerical interpretation domains are whole, equality is actual equality, and in both theories $x = y$ defines the numerical composite isomorphism. Contradiction translates to $0 = S0$, so the presentations are equiconsistent.*

*Proof.* Compose the exact comparisons with the explicit native arithmetic and logical interpretations. Normalization is internally equal to raw inclusion in the reverse direction, and the equality-rule changes are the identity on end sequents. The inherited numerical and formula round trips remain valid. The identity numerical relation is internally total, bijective, and preserves every interpreted operation. Falsity is preserved in both directions. □

This is a written mathematical proof extending the pinned formalized native bridge. The new restricted kernels and proof translations have not been checked by Lean or Rocq.

# 6 Complete implementations and reproducible comparison

## 6.1 The selected programs

The delivered checkers use Python's standard library only. Each acceptance module contains its own JSON reader, syntax and scope checks, capture-safe substitution, complete computation normalizer, proof-rule checks, final goal check, and output convention. There is no imported proof kernel, trusted theorem database, compressed payload, dynamic evaluation, or external normalization service.

| Complete module | Bytes | Tokens | AST nodes | Gzip-9 |
|---|---|---|---|---|
| Native transport, exact upstream | 2529 | 908 | 965 | 963 |
| Native newest-variable substitution | **2499** | **899** | **951** | 955 |
| Clausal transport, matched control | 2856 | 907 | 982 | 1061 |
| Clausal newest-variable substitution | **2826** | **898** | **968** | 1055 |

All measurements use CPython 3.12.14. The complete source is parsed with `ast.parse`, normalized with `ast.unparse`, and terminated by one line feed. Comments disappear under this operation. Significant-token counts omit encoding, end-marker, indentation, dedentation, newline and comment tokens. AST nodes are the nodes visited by `ast.walk`. The compared modules have no docstrings. Gzip is informational, with compression level 9 and timestamp zero.

The native control is a byte-for-byte copy of `full_normalization/goal_checker/code/full_expression_checker.py` at the pinned commit. Its source SHA-256 is

`869fc1ded752a0364284b9b4f53fddcc8b197200f8e5c73aed7bec55ca56252d`.

The selected native change saves 30 normalized bytes (about 1.19%), 9 significant tokens, and 14 AST nodes. It changes the equality packet and its checking branch; the original substitution and normalization implementation is unchanged. The clausal pair is a newly constructed matched pair with identical syntax, normalizer, clauses, and input contract. It saves the same 30 bytes, 9 tokens, and 14 nodes.

These are gains against the stated complete controls. The repository also contains much shorter arity-encoded programs, including sub-kilobyte validated profiles. The present programs do not establish a new global byte, token, or AST minimum across that heterogeneous corpus. The five-rule program is larger than the six-rule program because its explicit clause administration costs more in this encoding.

## 6.2 Native input and inference packets

The native checker accepts a JSON object with exactly `variables`, `context`, `goal`, and `proof`. Variables use zero-based de Bruijn indices. The numerical syntax is

| | |
|---|---|
| `["z"]` | $0$, |
| `["v",i]` | $v_i$, |
| `["s",t]` | $S(t)$, |
| `["i",n,a,h]` | $I(n, a; u.h)$. |

The update's state binder is index zero in $h$ and binds nowhere else. Formula tags are `"="`, `"q"`, and `"d"`. In the matrix of $D_{x,y}B$, index zero is $y$ and index one is $x$. Canonicality is checked, not

presumed: supplying an uncontracted computation in a goal or used term fails the canonical-syntax check. Boolean JSON values are not accepted as integer variable indices or scope counts.

The only changed packet is

`["vs",t,u,p,q]`.

It verifies $p$ against $v_0 = t$, verifies $q$ against $u = v$ where $v$ is the left endpoint of the requested goal, and checks that the right endpoint is $\mathrm{nf}(u[t/v_0])$. Substitution uses the existing nondropping operation: the current variable is replaced, while all other numerical parameters remain in scope. There is no new traversal.

Hypothesis and reflexivity are automatic goal checks. As in the exact upstream control, their unused proof hint is ignored. This is an explicit part of the certificate contract: the checker validates every requested goal and every used rule argument, but does not promise to validate irrelevant hint subtrees. The final claim is still checked. The remaining packets are `qe`, `de`, and `di`, with the same behavior as the source control.

### 6.3 Five-rule forward checker

The clausal checker accepts exactly `goal` and `proof`, for a closed Eq/$D$ theorem. It derives the conclusion from its proof tree and then compares it with the supplied goal. Its five packets are `r`, `x`, `u`, `e`, and `i`. The equality packet `["x",p,q]` has only two premise proofs. Their conclusions determine the replacement target and minor equation; the major source must be `["v",0]`. There is no supplied term template or supplied composite source.

Retained clauses are an implementation context, not a new formula constructor. Scope extension shifts both their antecedents and their consequents, protecting the consequent's own binder. The accompanying arbitrary-variable checker is retained as an auditable intermediate comparison and as a control for rejection of an older primitive source.

### 6.4 Verification evidence and its limits

The executable tests construct actual primitive proof trees for the direct transport simulation, rather than calling a trusted congruence operation. They exercise a zero witness, composite sources, repeated template occurrences, nested iterator bodies, passive occurrences of the source, and computations exposed by substitution.

The tests also nest translated transport proofs, carry the resulting proofs into extra I/E scopes, permute initial parameters, and replay the same derivations through the native goal checker. These cases address the newest-source invariant directly. An attempted primitive use of an older parameter is rejected by the newest-source kernel and accepted by the arbitrary-variable control.

The final evidence records 57 derivation and rejection checks, 804 independent direct-evaluation checks, and 74 command-line replays, all passing with no skipped semantic cases. The 804 semantic checks comprise 200 each for normalization, nondropping substitution, dropping substitution, and shifting, plus four explicit nested-binder capture checks. The CLI replays run both ordinary Python and optimization mode. Negative cases cover malformed syntax, scope escape, incorrect binder handling, and false conclusions. Reproduction requires no network access.

Finite tests do not prove bi-interpretability, normalization termination, or universal proof-translation correctness. Those are the mathematical arguments in this report. Conversely, the mathematical arguments do not by themselves certify the Python implementation; the source listings, matched

controls, proof compiler, and tests make that implementation reviewable. No new Lean or Rocq proof is claimed.

## 7 Further research questions

1. **Formalize the new proof translation.** Deeply embed the newest-variable kernel and prove the protected translation invariant, followed by exact equivalence with the normalized transport calculus. The original native/iPRA bridge can then be reused. The normalization layer must cover both computation rules, not just the already formalized literal-zero contraction.
2. **Transfer the reduction to the smallest arity codecs.** The mathematical change is independent of tagged JSON, but a fair measurement must count the entire validator and normalizer. The short raw codecs cannot simply inherit this theorem: the naming proof currently uses literal normal-form selector instances before general equality transport has been recovered.
3. **Reduce the proof expansion.** The direct simulation uses one new VSub inference and each supplied premise proof once. Its existential package adds logical scaffolding and enlarges repeated term labels. Compare tree and DAG certificate costs, including the complete checker, rather than moving an uncounted compiler into the trusted boundary.
4. **Identify the weakest normalization sufficient for naming.** The proof uses selector equations and two closed parity computations. Can a smaller, substitution-coherent normalization discipline enforce exactly those computations while retaining the same variable-source theorem? A few chosen rewrite abbreviations are insufficient unless their interaction with substitution and nested binders is specified.
5. **Determine a defensible lower bound.** A meaningful optimum needs a fixed implementation language, accepted certificate interface, primitive operations, validation requirements, and resource convention. In their absence, a smaller program is a comparison result rather than a lower bound on all possible axiomatizations.

## 8 Deliverable status

The completed result is the equivalence theorem for variable-source and newest-variable substitution, together with both ordinary iPRA composites, two complete newest-source checkers, their matched controls, and reproducible evidence. The arithmetic and logical bridge is attributed to the pinned project. The program-size improvement is modest and measured on the specified profiles; the primitive equality restriction is substantially stronger than an exclusion of a single source term.

## References

[1] V. Reshetnikov and ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, pinned commit `5d85023b0688e03b75f0ec3a04bb43ef0bb51705`. https://github.com/VladimirReshetnikov/ProveIt/blob/5d85023b0688e03b75f0ec3a04bb43ef0bb51705/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md.

[2] ProveIt, *Full computation normalization*, `Research/full_normalization/sections/normalization.tex` and `sections/calculus.tex`, at the pinned commit. Exact source copies are included in `reference/`.

[3] ProveIt, *Raw clausal computational equality*, `Research/raw_clausal/theory/equivalence.tex`, at the pinned commit. The clause representation and its logical interpretation are inherited.

[4] ProveIt, *Template-free equality in a seven-rule presentation of iPRA*, `Research/template_free/README.md` and its report, at the pinned commit. The arbitrary-source replacement rule and raw-marker argument are prior work.

[5] ProveIt, *Native scalar logic and the ordinary arithmetic bridge*, `Research/certified_zero/reference/`: `inherited_choice_basis.tex`, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex`, at the pinned commit. Exact source copies and the upstream license are included.

# A Selected complete native checker

This is the entire acceptance module used in the 2,499-byte normalized measurement. Its readable source includes the comments shown below.

```
# Complete standalone goal-directed newest-variable normal-form checker.
# See ../FORMAT.txt for syntax, proof packets, and the exact acceptance contract.
# No helper module is imported; exceptions reject; success is exactly ACCEPT.
import json
import sys


def require(condition):
    if not condition:
        raise ValueError('Invalid certificate')


def walk(a, f, d=0, sort=-1):
    require(type(a) is list and a)
    tag, *args = a
    require(sort < 0 or (tag in ('=', 'q', 'd')) == sort)
    if tag == 'v':
        require(len(args) == 1)
        return f(args[0], d)
    binders = {'z': [], 's': [0], 'i': [0, 0, 1], '=': [0, 0], 'q': [1], 'd': [2]}[tag]
    require(len(args) == len(binders))
    a = [tag] + [walk(b, f, d + k, tag in ('q', 'd')) for b, k in zip(args, binders)]
    return a[2] if a[:2] == ['i', ['z']] else sub(a[3], [tag, a[1][1], *a[2:]]) if tag == 'i' and a[1][0] == '
     s' else a


def syntax(a, n, sort=0):
    def variable(i, d):
        require(type(i) is int and 0 <= i < n + d)
        return ['v', i]
    require(walk(a, variable, sort=sort) == a)


def shift(a, k=1):
    return walk(a, lambda i, d: ['v', i + k if i >= d else i])
```

```
def sub(a, t, drop=1):
    return walk(a, lambda i, d: ['v', i] if i < d else shift(t, d) if i == d else ['v', i - drop])


def prove(a, p, g, n):
    syntax(a, n, 1)
    if a in g or a[0] == '=' and a[1] == a[2]:
        return
    match p:
        case ['qe', b, t, p, q]:
            syntax(t, n)
            prove(['q', b], p, g, n)
            require(a == sub(b, t))
            prove(sub(b, ['z']), q, g, n)
        case ['de', b, p, q]:
            prove(['d', b], p, g, n)
            prove(shift(a), q, [['q', b]] + [shift(x) for x in g], n + 1)
        case ['di', t, p]:
            require(a[0] == 'd')
            syntax(t, n)
            b = sub(['q', a[1]], t)[1]
            h = b if b[0] == '=' and t == ['z'] else shift(sub(b, ['z']))
            prove(sub(b, ['s', ['v', 0]], 0), p, [h] + [shift(x) for x in g], n + 1)
        case ['vs', t, u, p, q]:
            prove(['=', ['v', 0], t], p, g, n)
            require(a[0] == '=' and a[2] == sub(u, t, 0))
            prove(['=', u, a[1]], q, g, n)
        case _:
            raise ValueError('Invalid proof')


def check(c):
    require(type(c) is dict and set(c) == {'variables', 'context', 'goal', 'proof'})
    n, g, a, p = (c[k] for k in ['variables', 'context', 'goal', 'proof'])
    require(type(n) is int and n >= 0 and type(g) is list)
    for b in g:
        syntax(b, n, 1)
    prove(a, p, g, n)


if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except (ValueError, TypeError, IndexError, KeyError, RecursionError):
        sys.exit(1)
    print('ACCEPT')
```